



where to put raw queries to be included in other queries?

why "includes" though?

what about multilevel inheritance?

how the path within extends (or includes) is resolved?

given a table that have:

- primary keys
- score columns: score_1, score_2, ..., score_k
- date column

✓

now suppose that we have instead n tables each one that have:

- same primary keys
- score column

✓

note that the score column may have different name in each

within the trino engine the union all from the same table
does scan the table n times?

with all that, consider my real situation. models such that:

- are splitted in n tables (may be 1)
- each table can be long (may have one score column) or wide (may have one score column)

✓

can we have the same for spark?

summarize the context and what we have now

Here is a summary of the architecture and solutions we have built.

The Context

We designed a pure Python and Jinja rendering environment to dynamically generate SQL. The goal was to create modular, inheritable SQL components (similar to temporary views) and solve a specific data normalization problem: converting a heterogeneous mix of wide and long tables into a unified, long-format master dataset.

We established a project structure that uses `Environment` and `FileSystemLoader` to separate core logic, raw queries, and reusable macros, allowing for multilevel template inheritance.

What We Have Now

We have a complete, programmatic SQL generation pipeline optimized for big data engines.

1. The Python Data Structure

A single, succinct dictionary structure controls the generation logic. It maps each source table to its physical columns and their corresponding logical types, natively handling both wide ($k > 1$) and long ($k = 1$) tables.

Python



```
src_mappings = {  
    'tbl_alpha': [('math_val', 'math'), ('phys_val', 'physics')], # Wide  
    'tbl_beta':  [('chem_score', 'chemistry')]                  # Long  
}
```

2. The Jinja Engine

We developed two engine-specific Jinja macros that consume this dictionary. Both macros guarantee exactly **one table scan per source** by performing the column-to-row expansion entirely in memory, avoiding the massive I/O penalty of `UNION ALL`.

Target Engine	Jinja Implementation Strategy	SQL Mechanism
Trino	Unpacks tuples into two parallel arrays (<code>var</code> and <code>type</code>).	<code>CROSS JOIN</code> <code>UNNEST(ARRAY[], ARRAY[])</code>
Spark SQL	Injects list length (k) and flattens tuples into a comma-separated sequence.	<code>LATERAL VIEW stack(k, val, type)</code>

3. The Execution Flow

When you call the rendering script (`main.py`), Jinja injects the Python dictionary into the chosen macro. The engine iterates through the dictionary, dynamically adjusting the unpivot parameters for each table's width, and chains them together with a final `UNION ALL` . The result is a highly efficient, production-ready SQL string ready to be executed against your cluster.

Where would you like to take this architecture next?

Handle missing columns gracefully



Build an automated testing script



for simplicity consider spark from now on. suppose the final query we want to build is such that:

- reads from n tables, each one contributing with some columns (analysis)



To solve this in Spark, the most efficient approach is a **Star Schema pattern**.

We will treat the m score tables as your granular, unpivoted **Fact Table** (the center of the star), and the n analysis tables as your **Dimension Tables** that enrich the facts.

To make the joins highly efficient in Spark, the Jinja template will:

1. **Column Prune:** Generate CTEs for each analysis table that select *only* the primary keys and the requested columns. This prevents Spark from reading or shuffling unnecessary data.
2. **Use `USING` Joins:** By joining with the `USING (keys)` syntax, Spark automatically deduplicates the primary keys in the result, allowing us to safely use `SELECT *` in the final query without producing "ambiguous column" errors.

Here is how to structure it.

1. The Python Data Structure

We expand your Python dictionary to include both the analysis tables (mapping tables to the specific columns you want) and the score tables. (Note: I have folded `event_date` into the `pks` list since it acts as a join key).

Python



```
config = {
    'pks': ['user_id', 'session_id', 'event_date'],

    # n analysis tables (Dimension tables)
    'analysis_tables': {
        'tbl_demographics': ['age_group', 'country'],
        'tbl_engagement': ['total_clicks']
    },

    # m score tables (Fact tables)
    'score_tables': {
        'tbl_alpha': [('math_val', 'math'), ('phys_val', 'physics')],
        'tbl_beta': [('chem_score', 'chemistry')]
    }
}
```

2. The Jinja Template

We import your existing `spark_normalize` macro, execute it, and then loop through the analysis tables to build the optimized joins.

File: `templates/queries/master_analysis.sql`

Django



```
{% import "macros/spark_normalize.sql" as transform %}

WITH
-- 1. The Fact Table: Normalize all m score tables into one long table
unified_scores AS (
    {{ transform.spark_normalize(score_tables, pks) }}
),
```

```
-- 2. The Dimension Tables: Prune columns from the n analysis tables
{% for tbl, cols in analysis_tables.items() %}
{{ tbl }}_pruned AS (
    SELECT
        {{ pks | join(', ') }} ,
        {{ cols | join(', ') }}
    FROM {{ tbl }}
){% if not loop.last %},{% endif %}
{% endfor %}

-- 3. The Final Join: Enrich the scores using Spark's deduplicating USING clause
SELECT *
FROM unified_scores
{% for tbl in analysis_tables.keys() %}
LEFT JOIN {{ tbl }}_pruned
    USING ({{ pks | join(', ') }})
{% endfor %}
```

(Note: The `spark_normalize` macro will need a minor tweak from the previous version to use `pks` rather than a separate `dt_col` since the date is now integrated into the primary keys).

3. The Compiled Spark SQL

When rendered, this generates a highly readable, execution-optimized Spark SQL query. The `USING` clause cleanly merges the keys, meaning the final `SELECT *` yields exactly what you want: the keys, the scores, and the specific analysis columns, with zero duplication.

SQL



WITH

```
-- 1. The Fact Table: Normalize all m score tables into one long table
unified_scores AS (
    SELECT
        user_id, session_id, event_date,
        u.score,
        u.score_type
    FROM tbl_alpha
    LATERAL VIEW stack(
        2,
```

```

        'math', math_val, 'physics', phys_val
    ) u AS score_type, score

UNION ALL

SELECT
    user_id, session_id, event_date,
    u.score,
    u.score_type
FROM tbl_beta
LATERAL VIEW stack(
    1,
    'chemistry', chem_score
) u AS score_type, score
),

```

-- 2. The Dimension Tables: Prune columns from the n analysis tables

```

tbl_demographics_pruned AS (
    SELECT
        user_id, session_id, event_date,
        age_group, country
    FROM tbl_demographics
),
tbl_engagement_pruned AS (
    SELECT
        user_id, session_id, event_date,
        total_clicks
    FROM tbl_engagement
)

```

-- 3. The Final Join: Enrich the scores using Spark's deduplicating USING clause

```

SELECT *
FROM unified_scores
LEFT JOIN tbl_demographics_pruned
    USING (user_id, session_id, event_date)
LEFT JOIN tbl_engagement_pruned
    USING (user_id, session_id, event_date)

```

implement using trino

To implement this same Star Schema pattern in Trino, we keep the Python data structure and the main query template structurally identical to the Spark version. We only need to swap the Spark `stack()` macro for our Trino `UNNEST()` macro.

Like Spark, Trino natively supports the `USING` clause for joins, which perfectly deduplicates the join keys when you use `SELECT *`.

1. The Updated Trino Macro

We modify the `trino_normalize` macro we built earlier so that it uses the unified `pks` array (which now includes the date column) instead of treating the date column as a separate variable.

File: `templates/macros/trino_normalize.sql`

Django



```
{% macro trino_normalize(srcs, pks) %}
{% for tbl, maps in srcs.items() %}
SELECT
    {{ pks | join(', ') }} ,
    u.score,
    u.score_type
FROM {{ tbl }}
CROSS JOIN UNNEST(
    -- Array 1: Physical column references
    ARRAY[ {% for m in maps %} {{ m[0] }} {% if not loop.last %}, {% endif %} {% endif %} {% enc

    -- Array 2: Logical score_type literals
    ARRAY[ {% for m in maps %} '{{ m[1] }}' {% if not loop.last %}, {% endif %} {% endif %} {
) AS u(score, score_type)
{% if not loop.last %}
UNION ALL
{% endif %}
{% endfor %}
{% endmacro %}
```

2. The Main Query Template

The main query structure remains virtually identical to the Spark version, but calls the Trino macro.

File: templates/queries/master_analysis_trino.sql

Django



```
{% import "macros/trino_normalize.sql" as transform %}

WITH
-- 1. The Fact Table: Normalize all m score tables into one long table
unified_scores AS (
    {{ transform.trino_normalize(score_tables, pks) }}
),

-- 2. The Dimension Tables: Prune columns from the n analysis tables
{% for tbl, cols in analysis_tables.items() %}
{{ tbl }}_pruned AS (
    SELECT
        {{ pks | join(', ') }},
        {{ cols | join(', ') }}
    FROM {{ tbl }}
) {% if not loop.last %}, {% endif %}
{% endfor %}

-- 3. The Final Join: Enrich the scores
SELECT *
FROM unified_scores
{% for tbl in analysis_tables.keys() %}
LEFT JOIN {{ tbl }}_pruned
    USING ({{ pks | join(', ') }})
{% endfor %}
```

3. The Compiled Trino SQL

When you pass the Python configuration dictionary into this template, Jinja generates the highly optimized Trino SQL. The `CROSS JOIN UNNEST` ensures single-table scans for the facts, while the `USING` clause cleanly merges the dimensions.

WITH

-- 1. The Fact Table: Normalize all m score tables into one long table

```
unified_scores AS (  
    SELECT  
        user_id, session_id, event_date,  
        u.score,  
        u.score_type  
    FROM tbl_alpha  
    CROSS JOIN UNNEST(  
        ARRAY[math_val, phys_val],  
        ARRAY['math', 'physics']  
    ) AS u(score, score_type)  
  
    UNION ALL  
  
    SELECT  
        user_id, session_id, event_date,  
        u.score,  
        u.score_type  
    FROM tbl_beta  
    CROSS JOIN UNNEST(  
        ARRAY[chem_score],  
        ARRAY['chemistry']  
    ) AS u(score, score_type)  
)
```

-- 2. The Dimension Tables: Prune columns from the n analysis tables

```
tbl_demographics_pruned AS (  
    SELECT  
        user_id, session_id, event_date,  
        age_group, country  
    FROM tbl_demographics  
)  
tbl_engagement_pruned AS (  
    SELECT  
        user_id, session_id, event_date,  
        total_clicks  
    FROM tbl_engagement  
)
```

-- 3. The Final Join: Enrich the scores

SELECT *

```
FROM unified_scores
LEFT JOIN tbl_demographics_pruned
    USING (user_id, session_id, event_date)
LEFT JOIN tbl_engagement_pruned
    USING (user_id, session_id, event_date)
```

elaborate on using keyword

The `USING` keyword is a SQL syntax feature used in `JOIN` operations. It serves as a shorthand when the columns you are joining on have the **exact same name** in both tables.

However, its biggest advantage—and the reason it was critical in our Jinja architecture—is how it changes the behavior of `SELECT *`.

Here is a breakdown of why `USING` is so powerful in programmatic SQL generation compared to the standard `ON` clause.

The Problem with `ON`

When you join two tables with the standard `ON` clause, the SQL engine retains the join columns from *both* tables.

SQL



```
SELECT *
FROM unified_scores s
LEFT JOIN tbl_demographics_pruned d
    ON s.user_id = d.user_id
```

If you execute this, your result set will have **two** `user_id` columns (one from `s` and one from `d`).

If you try to write this to a new table, or wrap it in another CTE, the SQL engine will throw an "ambiguous column name" error because it doesn't know which `user_id` is which. To

fix it, you are forced to explicitly list out every single column you want, avoiding the duplicate:

SQL



```
-- Forced to be verbose
SELECT
    s.user_id, s.session_id, s.event_date, s.score,
    d.age_group, d.country
...
```

Writing out every column name manually defeats the purpose of dynamic Jinja templates.

The Magic of `USING`

When you use the `USING` keyword, the SQL engine (whether Trino, Spark, or PostgreSQL) does something special: it **coalesces (deduplicates)** the join keys.

SQL



```
SELECT *
FROM unified_scores
LEFT JOIN tbl_demographics_pruned
    USING (user_id, session_id, event_date)
```

Because you explicitly told the engine that these columns represent the exact same entity across both tables, `SELECT *` will output `user_id`, `session_id`, and `event_date` **only once**.

Why this is critical for our Jinja setup:

Because `USING` deduplicates the keys, we can confidently use a simple `SELECT *` at the end of our Jinja template.

We do not need to write complex Python logic to parse the `analysis_tables` dictionary, extract every single column name, and inject them into a massive `SELECT` block just to

avoid duplicates. The SQL engine handles the cleanup automatically.

ON vs. USING Summary

Feature	ON	USING
Syntax	<code>ON a.user_id = b.user_id</code>	<code>USING (user_id)</code>
Column Names	Can be different (e.g., <code>a.id = b.user_id</code>)	Must be identical in both tables
<code>SELECT *</code> Behavior	Returns duplicate join columns	Deduplicates join columns
Conditions	Supports inequalities (<code>></code> , <code><</code>) and <code>AND/OR</code>	Only supports equality (<code>=</code>)

When you cannot use USING

You are forced to fall back to the `ON` clause if:

1. The primary key in Table A is `user_id` , but in Table B it is `client_id` .
2. You need a non-equi join (e.g., `ON a.event_date > b.start_date`).

in that context, suppose the analysis tables does have different column names (for the same information) or some primary keys are missing and we need to retrieve from others. what is the best approach to it? normalize all first (such as saving in macros, includes or whatever) all tables?

Yes, you should absolutely normalize first. In data pipeline architecture, this is known as building a **Staging Layer**.

If you attempt to resolve missing keys, perform lookup joins, and rename columns dynamically inside the final Star Schema loop, your Jinja template will collapse under its own complexity, and you will lose the ability to use the efficient `USING` clause.

The most robust approach is to isolate the normalization of each messy analysis table into its own static component, and then use Jinja's `{% include %}` directive to inject them into the main query before the final automated join.

1. The Staging Layer (Includes)

Create a `staging/` directory in your Jinja project. For any table that deviates from the standard `pks` schema, write a succinct, raw SQL query that standardizes it.

File: `templates/staging/stg_demo.sql`

(Scenario: Renaming a primary key)

SQL



SELECT

```
    client_id AS user_id,  
    session_id,  
    event_date,  
    age_group
```

FROM raw_demographics

File: `templates/staging/stg_geo.sql`

(Scenario: Resolving a missing primary key via a lookup)

SQL



SELECT

```
    g.user_id,  
    l.session_id,  -- Retrieved from lookup  
    g.event_date,  
    g.country
```

FROM raw_geo g

```
LEFT JOIN session_lookup l
  ON g.user_id = l.user_id AND g.event_date = l.event_date
```

2. The Python Configuration

Update your Python dictionary to reference these staging files instead of the raw tables. You no longer need to pass the analysis columns explicitly, as the staging files will handle column selection.

Python



```
config = {
    'pks': ['user_id', 'session_id', 'event_date'],
    'analysis_files': ['stg_demo', 'stg_geo'],
    'score_tables': { ... } # Same as before
}
```

3. The Main Template

Update the main query to `include` these normalized files as Common Table Expressions (CTEs). Because every staging file guarantees the correct `pks` are present and named correctly, the automated `USING` join at the bottom remains perfectly clean.

File: `templates/queries/master_analysis.sql`

Django



```
{% import "macros/normalize.sql" as transform %}

WITH
-- 1. Fact Table
unified_scores AS (
    {{ transform.normalize(score_tables, pks) }}
),

-- 2. Dimension Tables (Injected via Includes)
{% for file in analysis_files %}
```



```

{{ file }} AS (
    {% include "staging/" ~ file ~ ".sql" %}
){% if not loop.last %},{% endif %}
{% endfor %}

-- 3. Final Automated Join
SELECT *
FROM unified_scores
{% for file in analysis_files %}
LEFT JOIN {{ file }}
    USING ({{ pks | join(', ') }})
{% endfor %}

```

Why this architecture wins:

1. **Separation of Concerns:** The complex logic of *cleaning* data (lookups, renames) is separated from the complex logic of *assembling* data (the Jinja join loops).
2. **Compactness:** The main Jinja template remains strictly structural and easy to read.
3. **Universality:** The final `SELECT * ... USING` block operates blindly, trusting that the staging layer has successfully conformed every table to the strict `pks` standard.

Want to map these staging files directly in Python?

↪ Yes

now, suppose each staging source table may be used with a variable number of primary keys or variable number of analysis columns. can we leverage this approach?

Yes, you can absolutely leverage this architecture to handle varying primary keys and columns. In fact, this is where Jinja truly outshines standard SQL.

If a dimension table only joins on `user_id`, but another joins on `user_id` and `session_id`, you cannot hardcode a single `USING (user_id, session_id)` clause. Furthermore, if you need to dynamically rename columns (`client_id` to `user_id`), creating dozens of static `.sql` staging files becomes tedious.

We can solve this entirely within the Python configuration by creating a **Universal Dimension Macro** that behaves exactly like our score normalization macro: metadata-driven and highly adaptable.

1. The Python Configuration (Physical-to-Logical Mapping)

Instead of just listing table names, we pass a dictionary that defines exactly which physical columns to extract and what their logical names should be. Notice how `tbl_geo` only provides one primary key, while `tbl_demographics` provides two.

Python



```
config = {
    # The master keys for the central Fact table
    'master_pks': ['user_id', 'session_id', 'event_date'],

    'dimensions': {
        'tbl_demographics': {
            'pks': [('client_id', 'user_id'), ('session_id', 'session_id')],
            'cols': [('age', 'age_group'), ('cntry', 'country')]
        },
        'tbl_geo': {
            # This table doesn't have a session_id, so we only join on user_id
            'pks': [('u_id', 'user_id')],
            'cols': [('region_code', 'region')]
        }
    }
}
```

2. The Universal Dimension Macro

We write a single macro that iterates through the Python dictionary to dynamically alias the columns for any dimension table.

File: `templates/macros/build_dimension.sql`

Django



```
{% macro build(src, config) %}
SELECT
    {% for phys, log in config.pks %}
    {{ phys }} AS {{ log }},
    {% endfor %}

    {% for phys, log in config.cols %}
    {{ phys }} AS {{ log }}{% if not loop.last %},{% endif %}
    {% endfor %}
FROM {{ src }}
{% endmacro %}
```

3. The Main Template (Dynamic `USING` Resolution)

This is where the magic happens. In the final `JOIN` block, we don't hardcode the `USING` clause. Instead, we instruct Jinja to look at the `pks` defined for that specific dimension in the Python dictionary, and build a custom `USING` clause containing only the provided logical keys.

File: `templates/queries/master_analysis.sql`

Django



```
{% import "macros/build_dimension.sql" as dim %}
{% import "macros/trino_normalize.sql" as fact %}

WITH
-- 1. The Fact Table
unified_scores AS (
    {{ fact.trino_normalize(score_tables, master_pks) }}
),

-- 2. The Dynamic Dimension Staging
{% for tbl, config in dimensions.items() %}
{{ tbl }}_stg AS (
    {{ dim.build(tbl, config) }}
){% if not loop.last %},{% endif %}
{% endfor %}

-- 3. The Adaptive Join
SELECT *
```

```

FROM unified_scores
{% for tbl, config in dimensions.items() %}
LEFT JOIN {{ tbl }}_stg
    -- Dynamically extract just the logical names for the USING clause
    USING ({% for phys, log in config.pks %}{{ log }}{% if not loop.last %}, {%
{% endfor %}}

```

4. The Compiled Output

When executed, Jinja perfectly adapts the query. It handles the aliasing inside the CTEs, and more importantly, it dynamically shrinks the `USING` clause for `tbl_geo` so it only attempts to join on `user_id`.

SQL



```

WITH
unified_scores AS (
    -- (Fact normalization logic generated here...)
),

-- 2. The Dynamic Dimension Staging
tbl_demographics_stg AS (
    SELECT
        client_id AS user_id,
        session_id AS session_id,
        age AS age_group,
        cntry AS country
    FROM tbl_demographics
),
tbl_geo_stg AS (
    SELECT
        u_id AS user_id,
        region_code AS region
    FROM tbl_geo
)

-- 3. The Adaptive Join
SELECT *
FROM unified_scores
LEFT JOIN tbl_demographics_stg
    USING (user_id, session_id)

```

```
LEFT JOIN tbl_geo_stg
  USING (user_id)
```

Handling the "Missing Key Lookup" Scenario

If a specific dimension table is so complex that a simple 1:1 alias mapping isn't enough (e.g., it requires an internal lookup join to retrieve a missing primary key), you can add a `type` flag to your Python dictionary.

Python



```
'tbl_complex': {
    'type': 'custom_sql',
    'file': 'staging/stg_complex.sql',
    'pks': ['user_id', 'session_id'] # Tell the main template what keys it outputs
}
```

Then, in your main Jinja loop, you simply add an `if` statement: if `type == 'custom_sql'`, use `{% include %}` to pull in the complex hand-written file. Otherwise, route it through the automated `build_dimension` macro.

give the detailed flow of all this

Here is the complete end-to-end execution flow of the architecture. This traces how a pure Python configuration dictionary is transformed into an optimized, execution-ready Spark SQL string without requiring verbose external frameworks.

1. State Definition (Python)

Execution begins in Python. You define the physical-to-logical mapping for all source systems using a compact dictionary. This state dictates the exact behavior of the Jinja compiler.

```
config = {
    'master_pks': ['user_id', 'session_id', 'event_date'],

    # Fact tables (Wide/Long to be normalized)
    'score_tables': {
        'tbl_alpha': [('math_val', 'math'), ('phys_val', 'physics')]
    },

    # Dimension tables (Varying PKs and columns to be pruned/aliased)
    'dimensions': {
        'tbl_demographics': {
            'pks': [('client_id', 'user_id'), ('sess', 'session_id')],
            'cols': [('age', 'age_group')]
        },
        'tbl_geo': {
            'pks': [('u_id', 'user_id')],
            'cols': [('region_code', 'region')]
        }
    }
}
```

2. Compilation Phase (Jinja Engine)

You pass the `config` dictionary into the Jinja `Environment`. The engine reads the main entry template (`master_analysis.sql`) and begins evaluating the logical blocks sequentially.

Step 2A: Fact Normalization

Jinja encounters the `spark_normalize` macro call.

It iterates over `score_tables`.

It calculates the length of the mapping array ($k = 2$ for `tbl_alpha`).

It generates a Common Table Expression (CTE) utilizing Spark's `LATERAL VIEW stack()` function, flattening the data in-memory during a single table scan.

Step 2B: Dimension Staging

Jinja loops through the `dimensions` dictionary and passes each table to the `build_dimension` macro.

It iterates through the physical-to-logical tuples.

It generates isolated CTEs (`tbl_demographics_stg` , `tbl_geo_stg`) that explicitly alias physical keys (`client_id`) to the logical master keys (`user_id`).

It drops any unmapped columns, ensuring Spark only reads necessary data.

Step 2C: Adaptive Joins

Jinja reaches the final `SELECT *` block.

It sets the normalized fact table (`unified_scores`) as the base.

It loops through the `dimensions` dictionary again.

Instead of blindly applying all `master_pks` , Jinja generates a highly targeted `USING` clause by extracting only the logical names defined in each dimension's specific `pks` array.

3. The Output (SQL Engine)

The Python script returns the compiled, mathematically sound SQL string. All Jinja loops and macros have collapsed into standard Spark SQL syntax.

SQL



```
WITH
-- [Phase 2A Output: Normalized Facts]
unified_scores AS (
    SELECT
        user_id, session_id, event_date,
        u.score, u.score_type
    FROM tbl_alpha
    LATERAL VIEW stack(
        2,
        'math', math_val, 'physics', phys_val
    ) u AS score_type, score
),
```

```

-- [Phase 2B Output: Staged Dimensions]
tbl_demographics_stg AS (
    SELECT
        client_id AS user_id,
        sess AS session_id,
        age AS age_group
    FROM tbl_demographics
),
tbl_geo_stg AS (
    SELECT
        u_id AS user_id,
        region_code AS region
    FROM tbl_geo
)

-- [Phase 2C Output: Deduplicating Joins]
SELECT *
FROM unified_scores
LEFT JOIN tbl_demographics_stg
    USING (user_id, session_id) -- Joins on 2 keys
LEFT JOIN tbl_geo_stg
    USING (user_id)             -- Joins on 1 key

```

4. Execution

The resulting string is passed to your Spark session (`spark.sql(query_string)`).

Because the query is structured as a Star Schema with pruned columns, in-memory unpivoting, and coalesced join keys, the Spark catalyst optimizer can seamlessly distribute the execution plan across the cluster with minimal shuffling.

now, suppose that also the score table have varying number of primary keys

When score tables have varying primary keys, we run into a hard SQL limitation: a `UNION ALL` requires every query to output the exact same columns in the exact same order.

If `tbl_alpha` has three primary keys and `tbl_beta` only has two, attempting to `UNION ALL` them will instantly crash the SQL engine.

To solve this, our Jinja macro must act as a **schema enforcer**. It will loop over a master list of required primary keys. If a score table has the key, it aliases it. If the score table is missing the key, the macro explicitly injects `NULL AS key_name` to maintain the structural integrity of the `UNION ALL`.

1. The Refined Python Structure

We will restructure the configuration slightly. By mapping the primary keys as a dictionary (`'logical_name': 'physical_name'`), we allow Jinja to perform fast, direct lookups to see if a key exists for a specific table.

Python



```
config = {
    # The ultimate schema for the unified fact table
    'master_pks': ['user_id', 'session_id', 'event_date'],

    'score_tables': {
        'tbl_alpha': {
            # Has all three master keys
            'pks': {'user_id': 'u_id', 'session_id': 'sess', 'event_date': 'dt'},
            'scores': [('math_val', 'math'), ('phys_val', 'physics')]
        },
        'tbl_beta': {
            # Missing 'session_id'
            'pks': {'user_id': 'client_id', 'event_date': 'evt_dt'},
            'scores': [('chem_score', 'chemistry')]
        }
    }
}
```

2. The Schema-Enforcing Macro

We update the `spark_normalize` macro. Notice the `if` statement iterating over the `master_pks`. It ensures that every `SELECT` block generates all three master columns regardless of the underlying table's actual schema.

Django



```
{% macro spark_normalize(score_tables, master_pks) %}
{% for tbl, config in score_tables.items() %}
SELECT
    -- 1. Enforce Master Primary Keys
    {% for logical_pk in master_pks %}
        {% if logical_pk in config.pks %}
            {{ config.pks[logical_pk] }} AS {{ logical_pk }},
        {% else %}
            NULL AS {{ logical_pk }},
        {% endif %}
    {% endfor %}

    -- 2. Unpivot the Scores
    u.score,
    u.score_type
FROM {{ tbl }}
LATERAL VIEW stack(
    {{ config.scores | length }},
    {% for phys, logical in config.scores %}'{{ logical }}', {{ phys }}{% if not
) u AS score_type, score

{% if not loop.last %}
UNION ALL
{% endif %}
{% endfor %}
{% endmacro %}
```

3. The Compiled SQL Output

When you run this through the Jinja compiler, it perfectly aligns the schemas.

Look closely at `tbl_beta` in the output below: because it was missing `session_id` in its Python configuration, Jinja safely injected `NULL AS session_id`.

SQL



```

WITH
unified_scores AS (
    SELECT
        u_id AS user_id,
        sess AS session_id,
        dt AS event_date,
        u.score,
        u.score_type
    FROM tbl_alpha
    LATERAL VIEW stack(
        2,
        'math', math_val, 'physics', phys_val
    ) u AS score_type, score

    UNION ALL

    SELECT
        client_id AS user_id,
        NULL AS session_id, -- Safely padded by Jinja
        evt_dt AS event_date,
        u.score,
        u.score_type
    FROM tbl_beta
    LATERAL VIEW stack(
        1,
        'chemistry', chem_score
    ) u AS score_type, score
)

... (Dimension CTEs and Final Join remain unchanged) ...

```

Why this seals the architecture

Because the fact table (`unified_scores`) is mathematically guaranteed to output the complete set of `master_pks` , the dimension joins we wrote in the previous step remain completely unchanged.

The dimension tables still dynamically dictate which keys they want to join on in their `USING` clause, and the central fact table is guaranteed to have those columns available, eliminating any risk of missing column errors during execution.

I mean, not within the scores but between scores and dimensions

Ah, I understand. You are talking about a **granularity mismatch** between the central Fact table (the scores) and the Dimension tables (the analysis).

For example, your unified score table might only be at the `user_id` level, but one of your dimension tables is tracked at the `user_id` AND `session_id` level. If you attempt a standard join using both keys, the SQL engine will crash with a "Column does not exist" error because `session_id` isn't in the score table.

To solve this, the Jinja compiler must dynamically calculate the **intersection** of the keys. It needs to look at the keys available in the score table, compare them to the keys in the dimension table, and safely build a `USING` clause that only includes the keys they both share.

Here is how you build that adaptive join logic.

1. The Python State

We define the primary keys available in the Fact table (`fact_pks`). We also define the dimensions as usual, but notice how `tbl_engagement` requires `session_id`, which does not exist in `fact_pks`.

Python



```
config = {
    # The unified scores table only has these two keys
    'fact_pks': ['user_id', 'event_date'],

    'dimensions': {
        'tbl_demographics': {
            # Shares both keys with the fact table
            'pks': [('client_id', 'user_id'), ('dt', 'event_date')],
            'cols': [('age', 'age_group')]
        },
        'tbl_engagement': {
            # Has a session_id, which the fact table DOES NOT have
            'pks': [('u_id', 'user_id'), ('sess', 'session_id')],
```

```

        'cols': [('clicks', 'total_clicks')]
    }
}

```

2. The Smart Join Template

We update the final `JOIN` block in your main Jinja template.

We will use a built-in Jinja utility called `joiner()`. It allows us to loop through the dimension keys, check if they exist in the `fact_pks`, and properly format the commas without trailing syntax errors.

File: `templates/queries/master_analysis.sql`

Django



```

SELECT *
FROM unified_scores

{% for tbl, dim_config in dimensions.items() %}
LEFT JOIN {{ tbl }}_stg
    -- Dynamically find the intersection of keys
    USING (
        {%- set comma = joiner(", ") -%}
        {%- for phys, logical in dim_config.pks -%}
            {%- if logical in fact_pks -%}
                {{ comma() }}{{ logical }}
            {%- endif -%}
        {%- endfor -%}
    )
{% endfor %}

```

3. The Compiled SQL Output

When Jinja compiles this, it acts as a safety valve. It successfully joins `tbl_demographics` on two keys. But when it reaches `tbl_engagement`, it detects that `session_id` is not present in the fact table, strips it out, and safely falls back to joining on `user_id` alone.

```
SELECT *  
FROM unified_scores  
  
LEFT JOIN tbl_demographics_stg  
  -- Intersection matched both keys  
  USING (user_id, event_date)  
  
LEFT JOIN tbl_engagement_stg  
  -- Intersection dropped session_id automatically  
  USING (user_id)
```

Important Data Consequence: By dropping `session_id` from the join condition, the SQL engine will match the single `user_id` from the score table against every session for that user in `tbl_engagement`. This is a one-to-many join, which will multiply the rows (fan-out) in your final dataset.

Want to prevent row duplication during these granular joins?

↔ Yes

Because dropping the `session_id` key causes a one-to-many join, how can I configure Jinja to automatically aggregate the dimension table before the join to prevent row fan-out?

To solve the fan-out problem, you must dynamically shift the grain of the dimension table *before* the join occurs.

We can instruct the Jinja compiler to look at the intersection of keys, drop the granular keys (like `session_id`), and automatically inject a `GROUP BY` clause with specified aggregate functions (like `SUM` or `MAX`) for the metrics.

Here is how to upgrade the architecture to support dynamic aggregation.

1. The Python State (Adding Aggregation Metadata)

We update the `cols` configuration to be a 3-tuple instead of a 2-tuple. The third element defines the mathematical aggregate function to apply when rolling up the data.

Python



```
config = {
    'fact_pks': ['user_id', 'event_date'],

    'dimensions': {
        'tbl_engagement': {
            'pks': [('u_id', 'user_id'), ('sess', 'session_id')],
            # Added 'SUM' and 'MAX' as the third tuple element
            'cols': [('clicks', 'total_clicks', 'SUM'), ('duration', 'max_durati
        }
    }
}
```

2. The Aggregating Dimension Macro

We update the `build_dimension.sql` macro.

To make this work, we use Jinja's `namespace()` object, which allows us to maintain a counter inside a `for` loop. We count exactly how many valid keys were kept, and use that integer to generate a positional `GROUP BY` (e.g., `GROUP BY 1, 2`).

File: `templates/macros/build_dimension.sql`

Django



```
{% macro build_aggregated_dimension(src, dim_config, fact_pks) %}
SELECT
    -- 1. Select ONLY the intersected keys
    {% set ns = namespace(key_count=0) %}
    {% for phys, logical in dim_config.pks %}
        {% if logical in fact_pks %}
        {{ phys }} AS {{ logical }},
        {% set ns.key_count = ns.key_count + 1 %}
    {% endfor %}
    FROM {{ src }}
```

```

        {% endif %}
    {% endfor %}

    -- 2. Apply the specific aggregation function to the metrics
    {% for phys, logical, agg_func in dim_config.cols %}
    {{ agg_func }}({{ phys }}) AS {{ logical }}{% if not loop.last %},{% endif %}
    {% endfor %}
FROM {{ src }}

-- 3. Dynamically generate the GROUP BY based on valid keys
{% if ns.key_count > 0 %}
GROUP BY {% for i in range(1, ns.key_count + 1) %}{{ i }}{% if not loop.last %},
{% endif %}
{% endif %}
{% endmacro %}

```

3. The Main Query

The main query passes the `fact_pks` into the dimension macro so it knows what to filter. The `USING` clause logic remains identical to the previous step.

File: `templates/queries/master_analysis.sql`

Django



```

{% import "macros/build_dimension.sql" as dim %}

WITH
unified_scores AS (
    -- (Fact logic unchanged)
),

-- 2. Dynamic Aggregation CTEs
{% for tbl, dim_config in dimensions.items() %}
{{ tbl }}_stg AS (
    {{ dim.build_aggregated_dimension(tbl, dim_config, fact_pks) }}
){% if not loop.last %},{% endif %}
{% endfor %}

-- 3. The 1-to-1 Join
SELECT *
FROM unified_scores
{% for tbl, dim_config in dimensions.items() %}

```



```

LEFT JOIN {{ tbl }}_stg
    USING (
        {% set comma = joiner(", ") -%}
        {% for phys, logical in dim_config.pks -%}
            {% if logical in fact_pks -%}
                {{ comma() }}{{ logical }}
            {% endif -%}
        {% endfor -%}
    )
{% endfor %}

```

4. The Compiled SQL Output

When the template compiles, Jinja identifies that `session_id` must be dropped. It successfully isolates `user_id` as the sole valid grouping key (`GROUP BY 1`), applies the `SUM` and `MAX` rollups to the metrics, and ensures the final join matches on `user_id` without duplicating rows.

SQL



```

WITH
unified_scores AS (
    -- ...
),

-- 2. Dynamic Aggregation CTEs
tbl_engagement_stg AS (
    SELECT
        u_id AS user_id,

        SUM(clicks) AS total_clicks,
        MAX(duration) AS max_duration
    FROM tbl_engagement
    GROUP BY 1
)

-- 3. The 1-to-1 Join
SELECT *
FROM unified_scores

```

```
LEFT JOIN tbl_engagement_stg
  USING (user_id)
```

what if we make available different versions of the same dimension table? e.g. aggregate a dimension before joining it. because some aggregations may use some sophisticated logic

To handle sophisticated, custom aggregations while maintaining the automated architecture, we need to introduce a **Router Pattern**.

Instead of forcing complex logic (like window functions, array aggregations, or conditional formatting) into a Python dictionary, we treat the dictionary as a dispatcher. It will route simple dimensions to our automated macro, and route complex dimensions to dedicated, hand-written SQL files.

This allows you to create multiple logical "versions" of the same physical table (e.g., a simple daily rollup vs. a complex 30-day rolling average) and join them simultaneously.

1. The Python State (The Dispatcher)

We update the configuration so the top-level keys are no longer physical table names, but **logical dimension names**. We also add a `type` flag to tell Jinja how to build it.

Python



```
config = {
    'fact_pks': ['user_id', 'event_date'],

    'dimensions': {
        # Version 1: Simple auto-aggregation (handled by Macro)
        'dim_engagement_simple': {
            'type': 'auto',
            'src_table': 'tbl_engagement',
            'pks': [('u_id', 'user_id')],
            'cols': [('clicks', 'total_clicks', 'SUM')]
```

```

    },

    # Version 2: Sophisticated logic (handled by Include)
    'dim_engagement_complex': {
        'type': 'custom',
        'file': 'dimensions/custom_engagement.sql',
        # We map logical-to-logical here to keep the Jinja USING loop uniform
        'pks': [('user_id', 'user_id'), ('event_date', 'event_date')]
    }
}
}
}

```

2. The Custom SQL File

For the complex version, you write pure Spark SQL in an isolated file. This file acts as a black box—the only requirement is that it outputs the logical primary keys defined in the Python dictionary.

File: templates/dimensions/custom_engagement.sql

SQL



```

SELECT
    u_id AS user_id,
    dt AS event_date,
    -- Sophisticated logic that would be awful to represent in a Python dictionary
    PERCENTILE_APPROX(session_duration, 0.95) AS p95_duration,
    COLLECT_SET(device_type) AS device_array,
    SUM(CASE WHEN is_bounced = false THEN 1 ELSE 0 END) / COUNT(1) AS retention_
FROM tbl_engagement
GROUP BY 1, 2

```

3. The Routing Template

We update the main Jinja loop to check the `type` flag. If it is `auto`, it calls the aggregation macro we built previously. If it is `custom`, it uses the `{% include %}` directive to inject your sophisticated file.

File: templates/queries/master_analysis.sql

```
{% import "macros/build_dimension.sql" as dim %}

WITH
unified_scores AS (
    -- ...
),

-- 2. The Router (Auto vs. Custom)
{% for dim_name, dim_config in dimensions.items() %}
{{ dim_name }} AS (
    {% if dim_config.type == 'auto' %}
        {{ dim.build_aggregated_dimension(dim_config.src_table, dim_config, fact
    {% elif dim_config.type == 'custom' %}
        {% include dim_config.file %}
    {% endif %}
){% if not loop.last %}, {% endif %}
{% endfor %}

-- 3. The Uniform Join
SELECT *
FROM unified_scores
{% for dim_name, dim_config in dimensions.items() %}
LEFT JOIN {{ dim_name }}
    USING (
        {%- set comma = joiner(", ") -%}
        {%- for phys, logical in dim_config.pks -%}
            {%- if logical in fact_pks -%}
                {{ comma() }}{{ logical }}
            {%- endif -%}
        {%- endfor -%}
    )
{% endfor %}
```

4. The Compiled SQL Output

Jinja successfully builds two completely different logical CTEs from the same underlying physical table, using two entirely different compilation methods. Because both methods output standardized logical keys, the final `USING` block blindly and successfully joins both versions to the Fact table.

```
WITH
unified_scores AS (
    -- ...
),

-- 2. The Router (Auto vs. Custom)
dim_engagement_simple AS (
    -- [Generated by the Macro]
    SELECT
        u_id AS user_id,
        SUM(clicks) AS total_clicks
    FROM tbl_engagement
    GROUP BY 1
),
dim_engagement_complex AS (
    -- [Injected verbatim from dimensions/custom_engagement.sql]
    SELECT
        u_id AS user_id,
        dt AS event_date,
        PERCENTILE_APPROX(session_duration, 0.95) AS p95_duration,
        COLLECT_SET(device_type) AS device_array,
        SUM(CASE WHEN is_bounced = false THEN 1 ELSE 0 END) / COUNT(1) AS retent
    FROM tbl_engagement
    GROUP BY 1, 2
)

-- 3. The Uniform Join
SELECT *
FROM unified_scores
LEFT JOIN dim_engagement_simple
    USING (user_id) -- Joined on 1 key
LEFT JOIN dim_engagement_complex
    USING (user_id, event_date) -- Joined on 2 keys
```

now, how to add a custom primary key that captures each score type? I mean, we may want to join scores with some dimension that match on score type (or some column that is one-to-one to this)

When you unpivot the score columns, you fundamentally change the **grain** of your Fact table. A single row no longer represents just a `user_id` and an `event_date` ; it now represents a `user_id` , an `event_date` , AND a `score_type` .

Because of this, `score_type` effectively becomes a new primary key for the `unified_scores` table.

To join a dimension on this new key (e.g., a metadata table that maps a `score_type` to a `department` or `weight_multiplier`), we just need to make a slight adjustment to our Python configuration. We must separate the keys used to *build* the Fact table from the keys available to *join* to the Fact table.

1. The Python State (Splitting the Keys)

We introduce `source_pks` (the keys that exist in the raw tables before unpivoting) and `fact_pks` (the keys that exist in the Fact table *after* unpivoting, which now includes `score_type`).

We also add a new dimension, `dim_score_meta` , which maps its physical column (`metric_name`) directly to our generated `score_type` .

Python



```
config = {
    # Keys used to pad/enforce the schema during the unpivot phase
    'source_pks': ['user_id', 'event_date'],

    # Keys available for dimensions to join on AFTER the unpivot phase
    'fact_pks': ['user_id', 'event_date', 'score_type'],

    'score_tables': {
        'tbl_alpha': {
            'pks': {'user_id': 'u_id', 'event_date': 'dt'},
            'scores': [('math_val', 'math'), ('phys_val', 'physics')]
```

```

    },
    'dimensions': {
        'dim_user': {
            'type': 'auto',
            'src_table': 'tbl_users',
            'pks': [('client_id', 'user_id')],
            'cols': [('tier', 'user_tier', 'MAX')]
        },
        # NEW: A dimension that joins specifically on the generated score_type
        'dim_score_meta': {
            'type': 'auto',
            'src_table': 'tbl_score_metadata',
            'pks': [('metric_name', 'score_type')],
            'cols': [('category', 'score_category', 'MAX'), ('weight', 'multipli
        },
    },
}

```

2. The Main Query Template

The beauty of the architecture we built is that **none of the dimension logic needs to change**.

We just pass `source_pks` into the fact-building macro (so it doesn't try to look for `score_type` in the raw tables), and we pass the expanded `fact_pks` into the dimension logic.

File: `templates/queries/master_analysis.sql`

Django



```

{% import "macros/spark_normalize.sql" as fact %}
{% import "macros/build_dimension.sql" as dim %}

WITH
-- 1. Build Facts (Using source_pks)
unified_scores AS (
    {{ fact.spark_normalize(score_tables, source_pks) }}
),

```

```

-- 2. Build Dimensions (Using fact_pks)
{% for dim_name, dim_config in dimensions.items() %}
{{ dim_name }} AS (
    {{ dim.build_aggregated_dimension(dim_config.src_table, dim_config, fact_pks) }}
    {% if not loop.last %}, {% endif %}
{% endfor %}

-- 3. Adaptive Join (Using fact_pks)
SELECT *
FROM unified_scores
{% for dim_name, dim_config in dimensions.items() %}
LEFT JOIN {{ dim_name }}
    USING (
        {%- set comma = joiner(", ") -%}
        {%- for phys, logical in dim_config.pks -%}
            {%- if logical in fact_pks -%}
                {{ comma() }}{{ logical }}
            {%- endif -%}
        {%- endfor -%}
    )
{% endfor %}

```

3. The Compiled SQL Output

When Jinja compiles the template, the intersection logic effortlessly detects that `dim_score_meta` shares `score_type` with the Fact table. It aggregates the metadata and joins it using the generated key, seamlessly enriching the dataset.

SQL



```

WITH
unified_scores AS (
    -- Unpivot generates the `score_type` column
    SELECT
        u_id AS user_id,
        dt AS event_date,
        u.score,
        u.score_type
    FROM tbl_alpha
    LATERAL VIEW stack(
        2,
        'math', math_val, 'physics', phys_val
    )

```



```

    ) u AS score_type, score
),

-- 2. Build Dimensions
dim_user AS (
    SELECT
        client_id AS user_id,
        MAX(tier) AS user_tier
    FROM tbl_users
    GROUP BY 1
),
dim_score_meta AS (
    -- Maps the physical 'metric_name' to the logical 'score_type'
    SELECT
        metric_name AS score_type,
        MAX(category) AS score_category,
        MAX(weight) AS multiplier
    FROM tbl_score_metadata
    GROUP BY 1
)

-- 3. Adaptive Join
SELECT *
FROM unified_scores
LEFT JOIN dim_user
    USING (user_id)                -- Joins on user
LEFT JOIN dim_score_meta
    USING (score_type)            -- Joins exclusively on the generated score type

```

note that the dim table may use the score_type and other primary keys also

This is exactly why we built the intersection logic dynamically. Because we separated source_pks from fact_pks in the previous step, the architecture already supports this natively without changing a single line of Jinja code.

If you have a highly granular dimension—for example, a table tracking a specific user's target for a specific score type on a specific day—you simply declare all those keys in the Python dictionary.

The Jinja compiler will mathematically intersect them, group them, and join them flawlessly.

1. The Python State (Multi-Key Dimension)

We add a new dimension (`dim_daily_targets`) that requires three keys: user, date, and the score type.

Python



```
config = {
    'source_pks': ['user_id', 'event_date'],
    'fact_pks': ['user_id', 'event_date', 'score_type'], # The master intersecti

    'score_tables': {
        'tbl_alpha': {
            'pks': {'user_id': 'u_id', 'event_date': 'dt'},
            'scores': [('math_val', 'math')]
        }
    },

    'dimensions': {
        # NEW: A dimension joining on user, date, AND score_type
        'dim_daily_targets': {
            'type': 'auto',
            'src_table': 'tbl_user_targets',
            'pks': [
                ('client_id', 'user_id'),
                ('target_date', 'event_date'),
                ('metric_name', 'score_type')
            ],
            'cols': [('target_value', 'daily_target', 'MAX')]
        }
    }
}
```

2. The Compiled SQL Output

Because the macro loops through the dimension's `pks` and checks if they exist in `fact_pks`, it successfully identifies all three keys. It increments the `GROUP BY` counter to 3, and generates a perfect `USING` clause for the multi-key join.

SQL



```
WITH
unified_scores AS (
  -- [Fact Generation]
  SELECT
    u_id AS user_id,
    dt AS event_date,
    u.score,
    u.score_type
  FROM tbl_alpha
  LATERAL VIEW stack(
    1,
    'math', math_val
  ) u AS score_type, score
),

-- [Dimension Aggregation]
dim_daily_targets AS (
  SELECT
    client_id AS user_id,
    target_date AS event_date,
    metric_name AS score_type,

    MAX(target_value) AS daily_target
  FROM tbl_user_targets

  -- Dynamically grouped by all 3 matching keys
  GROUP BY 1, 2, 3
)

-- [Final Adaptive Join]
SELECT *
FROM unified_scores
LEFT JOIN dim_daily_targets
  -- Dynamically joined on all 3 matching keys
  USING (user_id, event_date, score_type)
```

By decoupling the metadata from the rendering engine, the Python dictionary acts as a strict mathematical mapping. The Jinja templates are reduced to pure algebraic operators (unpivot, intersect, group, join), capable of handling any combination of table widths or granularities you throw at them.

now, summerize the detailed flow again, illustrating with example

Here is the complete, end-to-end flow of the automated Star Schema generator. This architecture transforms a declarative Python dictionary into an execution-ready, highly optimized Spark SQL string, seamlessly handling missing keys, dynamic unpivoting, and varying table granularities.

Step 1: The Python State (The Blueprint)

Execution begins in Python. You define a single dictionary that maps the messy physical reality of your tables to a clean, logical schema.

In this example, we have a wide fact table missing the `session_id`, a dimension joining on just the user, and a highly granular dimension joining on user, date, and the generated `score_type`.

Python



```
config = {
    # Keys for reading raw tables
    'source_pks': ['user_id', 'session_id', 'event_date'],
    # Keys available AFTER unpivoting
    'fact_pks': ['user_id', 'session_id', 'event_date', 'score_type'],

    'score_tables': {
        'tbl_alpha': {
            # Missing 'session_id' in the raw table
            'pks': {'user_id': 'u_id', 'event_date': 'dt'},
            'scores': [('math_val', 'math'), ('phys_val', 'physics')]
        }
    }
}
```

```

    },
    'dimensions': {
        'dim_user_profile': {
            'type': 'auto',
            'src_table': 'tbl_users',
            # Joins only on user_id
            'pks': [('client_id', 'user_id')],
            'cols': [('tier', 'user_tier', 'MAX')]
        },
        'dim_daily_targets': {
            'type': 'auto',
            'src_table': 'tbl_targets',
            # Joins on user, date, AND score_type
            'pks': [
                ('u_id', 'user_id'),
                ('tgt_date', 'event_date'),
                ('metric', 'score_type')
            ],
            'cols': [('val', 'target_value', 'SUM')]
        }
    }
}

```

Step 2: The Jinja Compilation Phase (The Engine)

You pass this dictionary into the Jinja environment. The compiler evaluates the templates in three distinct logical phases:

1. Fact Normalization (Schema Enforcement & Unpivot):

Jinja iterates over `score_tables`. It cross-references the table's physical keys against `source_pks`. Because `tbl_alpha` is missing `session_id`, Jinja injects `NULL AS session_id` to guarantee schema integrity. It then counts the number of score columns (2) and generates the `LATERAL VIEW stack()` command to unpivot the data in-memory during a single table scan.

2. Dimension Aggregation (Intersection & Rollup):

Jinja iterates over `dimensions`. For each table, it checks which of its logical keys exist in the `fact_pks` master list. It counts the valid keys, aliases them, applies the requested aggregation functions (`MAX` , `SUM`) to the metrics, and injects a dynamic `GROUP BY` clause matching the exact number of valid keys. This guarantees no row fan-out will occur during the join.

3. Adaptive Assembly (The `USING` Generator):

In the final `SELECT *` block, Jinja loops through the dimensions one last time. It recalculates the key intersection and generates a targeted `USING (key1, key2)` clause for each dimension.

Step 3: The Compiled Spark SQL (The Output)

The Jinja templates collapse, returning a mathematically sound, deduplicated SQL string perfectly formatted for Spark's catalyst optimizer.

SQL



WITH

-- [Phase 1: Fact Normalization]

```
unified_scores AS (  
  SELECT  
    u_id AS user_id,  
    NULL AS session_id, -- Safely padded missing key  
    dt AS event_date,  
    u.score,  
    u.score_type  
  FROM tbl_alpha  
  LATERAL VIEW stack(  
    2,  
    'math', math_val, 'physics', phys_val  
  ) u AS score_type, score  
) ,
```

-- [Phase 2: Dimension Aggregation]

```
dim_user_profile AS (  
  SELECT  
    client_id AS user_id,  
    MAX(tier) AS user_tier  
  FROM tbl_users  
  GROUP BY 1 -- Grouped by 1 key  
) ,  
dim_daily_targets AS (  
  SELECT  
    u_id AS user_id,  
    tgt_date AS event_date,  
    metric AS score_type,  
    SUM(val) AS target_value  
  FROM tbl_targets
```

```

GROUP BY 1, 2, 3 -- Grouped by 3 keys
)

-- [Phase 3: Adaptive Assembly]
SELECT *
FROM unified_scores
LEFT JOIN dim_user_profile
    USING (user_id) -- Joins on 1 key
LEFT JOIN dim_daily_targets
    USING (user_id, event_date, score_type) -- Joins on 3 keys

```

Step 4: Execution

You pass the resulting string directly to your cluster via `spark.sql(query_string)`. The engine processes a perfectly formed Star Schema, deduplicating the join columns natively and reading each raw table exactly once.

consider that the situation in with the dimension tables have somewhat complex logic is the default. also:

- the final user should have the possibility to easily add new dimension and score configs

If complex logic is the default, relying heavily on a massive Python dictionary becomes an anti-pattern. It forces data engineers to constantly modify core application state just to add a new SQL metric, which is risky and creates a bottleneck.

Furthermore, introducing parameterized logic (like time delays) breaks the automated `USING (keys) join`, because joining on an offset requires an explicit `ON fact.date = dim.date - INTERVAL 7 DAYS`.

To solve this, we must pivot from a "Python-driven" architecture to a "**Configuration-Driven Component**" architecture. Here are the four major improvements to upgrade your workflow.

1. Shift to YAML Configuration (For Easy Onboarding)

Instead of hardcoding the state in Python, move it to YAML. Analysts and end-users are highly comfortable modifying YAML.

By separating the configuration from the execution script, adding a new dimension is as easy as adding a block of text to a file.

File: `config.yml`

YAML



```
dimensions:
  # The rare simple case
  dim_user_profile:
    type: auto
    src_table: tbl_users
    pks:
      user_id: client_id
    cols:
      user_tier: MAX(tier)

  # The default complex case
  dim_churn_prediction:
    type: custom
    file: dimensions/churn_model.sql
    pks: [user_id, event_date]
    params:
      lookback_days: 30
      prediction_window: 7
```

Python Improvement: Your Python script now just does `config = yaml.safe_load(open('config.yml'))` and passes it to Jinja.

2. Context Injection for Parameterized SQL

Because complex logic is the default, your end-users will be writing a lot of custom `.sql` files. To support parameterization, we can leverage Jinja's variable scoping.

When the router uses `{% include %}`, the included file inherits all variables currently set in the Jinja context. We can explicitly unpack the `params` from the YAML and inject them into the `.sql` file.

The Router Update:

Django



```
{% if dim_config.type == 'custom' %}
    -- Unpack parameters into the Jinja context before including
    {% set params = dim_config.params | default({}) %}
    {% include dim_config.file %}
{% endif %}
```

File: `dimensions/churn_model.sql` (Written by the end-user)

SQL



```
SELECT
    user_id,
    event_date,
    SUM(clicks) OVER (
        PARTITION BY user_id
        ORDER BY event_date
        -- The analyst seamlessly uses the YAML parameters
        ROWS BETWEEN {{ params.lookback_days }} PRECEDING AND CURRENT ROW
    ) as historical_engagement
FROM tbl_events
```

3. Upgrade to an `ON` Clause Generator

If an end-user wants to join a dimension with a delay (e.g., joining today's score with a dimension row from 7 days ago), the `USING (event_date)` shortcut fails. We must upgrade the final join block to explicitly write `ON` conditions.

We update the YAML to allow an optional `join_expression` for specific keys.

In `config.yml`:

YAML



```

dim_delayed_target:
  type: custom
  file: dimensions/targets.sql
  pks:
    - user_id
    - event_date
  join_conditions:
    # Explicitly define how this specific key should join to the fact table
    event_date: "fact.event_date = dim.event_date + INTERVAL '7' DAY"

```

The Final Join Update:

Instead of `USING`, the Jinja template dynamically compiles the `ON` clause, defaulting to an equals sign unless overridden by the YAML.

Django



```

SELECT *
FROM unified_scores AS fact
{% for dim_name, dim_config in dimensions.items() %}
LEFT JOIN {{ dim_name }} AS dim
  ON 1=1
  {% for logical_key in dim_config.pks %}
    {% if logical_key in fact_pks %}
      -- Check if a custom join condition was provided for this key
      {% if dim_config.join_conditions and logical_key in dim_config.join_
        AND {{ dim_config.join_conditions[logical_key] }}
      {% else %}
        -- Default to standard equi-join
        AND fact.{{ logical_key }} = dim.{{ logical_key }}
      {% endif %}
    {% endif %}
  {% endif %}
{% endfor %}

```

4. Auto-Discovery (Drop-and-Go Workflow)

To completely remove friction for the end-user, eliminate the need to register custom dimensions in the YAML altogether.

You can update your Python script to automatically scan the `templates/dimensions/` directory using Python's `os` or `pathlib`.

1. An analyst drops a new complex query into `templates/dimensions/dim_fraud_score.sql`.
2. They place a small header at the very top of the SQL file using Jinja comments to declare their keys: `{%# keys: [user_id, event_date] #}`.
3. The Python script parses the directory, reads the headers, auto-populates the dictionary, and passes it to the Jinja compiler.

The result: The end-user never touches Python, never touches YAML for custom queries, and only writes raw, parameterized SQL. The engine dynamically stages it, calculates the key intersection, and attaches it to the Star Schema automatically.

elaborate on 4

The "Auto-Discovery" or "Drop-and-Go" workflow is inspired by modern data tools like dbt. It solves the biggest bottleneck in data engineering: centralized configuration files.

If every new metric requires an engineer to update a massive, 1,000-line YAML or Python dictionary, the system becomes fragile, and pull requests get bottlenecked.

With Auto-Discovery, we decentralize the configuration. The Python script dynamically inspects the file system at runtime, reads metadata embedded directly inside the SQL files, and builds the Star Schema automatically.

Here is exactly how to implement this architecture.

1. The Developer Experience (What the User Does)

When a data analyst wants to add a new, highly complex dimension (e.g., a fraud scoring model) to the master dataset, they do not touch Python, and they do not touch the central YAML file.

They simply create a new file in the `templates/dimensions/` directory. At the top of their SQL file, they write a Jinja comment block containing a tiny snippet of YAML. This is called **Frontmatter**.

File: templates/dimensions/dim_fraud_score.sql

SQL



```
{# config
pks:
  - user_id
  - event_date
join_conditions:
  event_date: "fact.event_date = dim.event_date + INTERVAL '1' DAY"
params:
  threshold: 0.85
#}

SELECT
  u_id AS user_id,
  dt AS event_date,
  MAX(model_confidence) AS max_confidence,
  -- Using the param defined in the frontmatter
  SUM(CASE WHEN model_confidence > {{ params.threshold }} THEN 1 ELSE 0 END) /
FROM enterprise_fraud_logs
GROUP BY 1, 2
```

To the end-user, this is incredibly intuitive. The configuration lives *exactly* where the logic lives.

2. The Python Parser (The "Discovery" Script)

Before initializing the Jinja environment, your Python script scans the `dimensions/` folder. It looks for `.sql` files, extracts the `{# config ... #}` block using a simple regular expression, and parses it into a Python dictionary.

File: main.py

Python



```
import os
import re
import yaml
```

```

from pathlib import Path

def discover_dimensions(directory_path: str) -> dict:
    dimensions = {}

    # Regex to capture everything between {# config and #}
    frontmatter_pattern = re.compile(r'{#\s*config\s*(.*?)\s*#}', re.DOTALL)

    for filepath in Path(directory_path).glob('*.sql'):
        table_name = filepath.stem # e.g., 'dim_fraud_score'
        sql_content = filepath.read_text()

        match = frontmatter_pattern.search(sql_content)
        if match:
            # Parse the extracted YAML text into a Python dictionary
            config_dict = yaml.safe_load(match.group(1))

            # Attach the system path and type so Jinja knows how to route it
            config_dict['type'] = 'custom'
            config_dict['file'] = f"dimensions/{filepath.name}"

            dimensions[table_name] = config_dict

    return dimensions

# 1. Auto-discover all custom SQL dimensions
dynamic_dimensions = discover_dimensions("templates/dimensions")

# 2. (Optional) Merge with a small config.yml containing your rare "auto" tables
# final_dimensions = {**static_dimensions, **dynamic_dimensions}

# 3. Pass to Jinja as usual
# html = template.render(dimensions=dynamic_dimensions, fact_pks=...)

```

3. How the Engine Reacts

When Python passes this auto-generated `dimensions` dictionary to Jinja, the main query template behaves exactly as we designed it in the previous steps.

1. **Routing:** It sees `type: custom` and uses `{% include "dimensions/dim_fraud_score.sql" %}` to inject the CTE.
2. **Variable Injection:** It unpacks `params.threshold = 0.85` and resolves the Jinja tags inside the analyst's custom query.

3. **Adaptive Join:** It looks at the `pks` , intersects them with the Fact table keys, checks the `join_conditions` dictionary, and generates the delayed `ON` clause natively.

Why this completes the architecture:

Zero-Friction Scaling: You can scale from 5 dimension tables to 500 dimension tables without writing a single line of Python or Jinja.

Encapsulation: If a query breaks, the error is isolated to that specific `.sql` file.

Code Review Clarity: When an analyst submits a pull request to add a metric, the reviewer only has to look at one file (`dim_fraud_score.sql`) to understand both the SQL logic and how it attaches to the wider data model.

within that, can the user also use an include and overwrite the default parameters?

Yes. This is where the architecture evolves from a simple SQL generator into a true **Component Library**.

Because Jinja executes sequentially and passes context down the chain, users can easily override parameters. However, to make this work, we have to write the base template slightly differently so it knows how to "listen" for those overrides.

Here are the two ways a user can override parameters, depending on whether they are building a nested component or just instantiating a new dimension in the Star Schema.

The Prerequisite: Making the Base File "Override-Aware"

When you use the Drop-and-Go frontmatter, the Python script reads the defaults. But if we want Jinja to respect dynamic overrides at runtime, the base SQL file must use Jinja's `default` filter or conditional logic to check if a parameter was already injected before falling back to the default.

File: `templates/dimensions/dim_fraud_base.sql`

SQL



```
{# config
pks: [user_id, event_date]
params:
    threshold: 0.85
#}

-- Safely unpack: Use the injected param if it exists, otherwise use 0.85
{% set threshold = params.threshold | default(0.85) if params is defined else 0.

SELECT
    user_id,
    event_date,
    SUM(CASE WHEN model_confidence > {{ threshold }} THEN 1 ELSE 0 END) AS high_
FROM enterprise_fraud_logs
GROUP BY 1, 2
```

(Note a beautiful synergy here: `{# ... #}` is a native Jinja comment. When Python reads the file, it extracts the config. But when Jinja compiles the file, it completely ignores the frontmatter block, meaning it won't leak garbage text into your final SQL).

Scenario 1: The "Jinja Include" Override (Nested Logic)

Suppose an analyst wants to write a brand new, highly customized query (`dim_strict_fraud.sql`), but they want to reuse the logic from the base fraud file as a Common Table Expression (CTE) inside their file.

They can use the `{% set %}` directive immediately before the `{% include %}` directive. Jinja pushes the newly set `params` object into the context of the included file.

File: `templates/dimensions/dim_strict_fraud.sql`

SQL



```
{# config
pks: [user_id, event_date]
#}
```

```

WITH strict_base AS (
    -- 1. Override the parameter
    {% set params = {'threshold': 0.99} %}

    -- 2. Include the file (it will now compile using 0.99)
    {% include 'dimensions/dim_fraud_base.sql' %}
)

SELECT
    user_id,
    event_date,
    high_risk_flags,
    -- Add net-new logic on top of the included file
    CASE WHEN high_risk_flags > 5 THEN 'LOCK_ACCOUNT' ELSE 'PASS' END AS action
FROM strict_base

```

Scenario 2: The "Config Alias" Override (Multiple Instantiations)

Suppose you don't need to add net-new logic; you just want **two versions** of the fraud dimension attached to your final Star Schema simultaneously (e.g., `fraud_standard` and `fraud_strict`).

Instead of writing a new Jinja file, the user simply adds a block to the central `config.yml` to alias the physical file and inject new parameters.

In `config.yml`:

YAML



```

dimensions:
  # Python reads this, ignores the file's frontmatter defaults,
  # and pushes THIS param dict into the Jinja context during the loop.
  dim_fraud_strict:
    type: custom
    file: dimensions/dim_fraud_base.sql
    pks: [user_id, event_date]
    params:
      threshold: 0.99

```


The Router Update

To support Scenario 2, your main router loop (in `master_analysis.sql`) just needs to inject the `params` dictionary into the context immediately before calling the include.

File: `templates/queries/master_analysis.sql`

Django



```
{% for dim_name, dim_config in dimensions.items() %}
{{ dim_name }} AS (

    {% if dim_config.type == 'auto' %}
        {{ dim.build_aggregated_dimension(...) }}

    {% elif dim_config.type == 'custom' %}
        -- Inject the specific params for this iteration of the dimension
        {% set params = dim_config.params | default({}) %}

        -- Compile the file using the injected params
        {% include dim_config.file %}
    {% endif %}

){% if not loop.last %},{% endif %}
{% endfor %}
```

The Result

You now have a system where a single `.sql` file can be executed with its default frontmatter configuration, embedded and overridden within another analyst's custom script, or cloned multiple times in the Star Schema with different thresholds—all without duplicating a single line of actual SQL logic.

note that, the usual workflow in mind is that:

- user define a set of scores (as we did in unifying section)
- user define which dim he wants to attach, as well which columns from each dim and which parameters for



To perfectly align with this workflow, we need to formally separate the architecture into two distinct concepts: a **Dimension Catalog** (the library of available logic) and a **User Runbook** (the configuration where the user explicitly requests what they want).

The biggest challenge in your described workflow is **column selection**. If a custom SQL file outputs 50 columns, but the user only requests 2, injecting the file directly into the CTE will expose all 50 columns.

To solve this, we can improve the Jinja router by wrapping the user's requested dimension in an **Outer Pruning Subquery**. This guarantees that the final join only attaches the exact columns the user asked for.

Here is how to upgrade the architecture for this "Catalog + Runbook" workflow.

1. The Dimension Catalog (The Library)

Advanced analysts and data engineers populate a repository with `.sql` files. These files act as black-box definitions. They output a standard set of keys and a wide array of potential analysis columns. They can use Jinja variables `{{ params.X }}` internally.

File: `templates/catalog/dim_fraud.sql`

SQL



```
SELECT
    user_id,
    event_date,
    SUM(CASE WHEN confidence > {{ params.threshold | default(0.5) }} THEN 1 ELSE
    AVG(confidence) AS avg_fraud_score,
    MAX(device_ip) AS last_known_ip
FROM raw_fraud_logs
GROUP BY 1, 2
```

2. The User Runbook (The Configuration)

This is the only piece the final user interacts with. They define their scores, and then they define an array of `enrichments`.

For each enrichment, they pick a dimension from the catalog, pass their desired parameters, and strictly define the columns they want returned. They can even alias the output if they want to call the same dimension twice with different parameters!

File: `user_runbook.yml`

YAML



```
master_pks: [user_id, event_date]

scores:
  tbl_alpha:
    pks: {user_id: u_id, event_date: dt}
    scores: [[math_val, math]]

enrichments:
  # The user wants fraud data, but ONLY the high_risk_flags column, using a strict threshold
  - instance_name: fraud_strict
    catalog_ref: dim_fraud
    pks: [user_id, event_date]
    params:
      threshold: 0.95
    columns:
      - high_risk_flags

  # The user calls the exact same catalog file again, but with a loose threshold
  - instance_name: fraud_loose
    catalog_ref: dim_fraud
    pks: [user_id, event_date]
    params:
      threshold: 0.50
    columns:
      - high_risk_flags
      - avg_fraud_score
```

3. The Outer-Pruning Jinja Template

We upgrade the main generation script. When iterating through the user's `enrichments`, Jinja uses the `catalog_ref` to fetch the file, injects the `params`, and places the entire include inside an inner subquery. The outer query then explicitly selects ONLY the `pks` and the requested `columns`.

Django



```
{% import "macros/spark_normalize.sql" as fact %}

WITH
-- 1. Build Fact Table (The User's Scores)
unified_scores AS (
    {{ fact.spark_normalize(scores, master_pks) }}
),

-- 2. Build the Requested Enrichments (With Outer Pruning)
{% for enrichment in enrichments %}
{{ enrichment.instance_name }} AS (
    SELECT
        -- Select the keys
        {{ enrichment.pks | join(', ') }} ,

        -- Select ONLY the columns the user explicitly requested
        {{ enrichment.columns | join(', ') }}
    FROM (
        -- Inject parameters and evaluate the catalog file
        {% set params = enrichment.params | default({}) %}
        {% include "catalog/" ~ enrichment.catalog_ref ~ ".sql" %}
    ) AS inner_catalog_query
){% if not loop.last %},{% endif %}
{% endfor %}

-- 3. The Adaptive Assembly
SELECT *
FROM unified_scores
{% for enrichment in enrichments %}
LEFT JOIN {{ enrichment.instance_name }}
    USING (
        {% - set comma = joiner(", ") -%}
        {% - for key in enrichment.pks -%}
            {% - if key in master_pks -%}
                {{ comma() }}{{ key }}
            {% - endif -%}
        {% - endfor -%}
    )
{% endfor %}
```

```
)  
{% endfor %}
```

4. The Compiled Output

The generated SQL is exactly what the user envisioned. The framework safely instantiated `dim_fraud.sql` twice, injected the correct parameters into the innermost scope of each, pruned away unrequested columns like `last_known_ip`, and joined them seamlessly.

SQL



```
WITH  
unified_scores AS (  
    -- [Fact Logic Unchanged]  
)  
  
-- 2. Build the Requested Enrichments  
fraud_strict AS (  
    SELECT  
        user_id, event_date,  
        high_risk_flags  
    FROM (  
        SELECT  
            user_id,  
            event_date,  
            SUM(CASE WHEN confidence > 0.95 THEN 1 ELSE 0 END) AS high_risk_flag,  
            AVG(confidence) AS avg_fraud_score,  
            MAX(device_ip) AS last_known_ip  
        FROM raw_fraud_logs  
        GROUP BY 1, 2  
    ) AS inner_catalog_query  
)  
fraud_loose AS (  
    SELECT  
        user_id, event_date,  
        high_risk_flags, avg_fraud_score  
    FROM (  
        SELECT  
            user_id,  
            event_date,
```

```

        SUM(CASE WHEN confidence > 0.50 THEN 1 ELSE 0 END) AS high_risk_flag
        AVG(confidence) AS avg_fraud_score,
        MAX(device_ip) AS last_known_ip
    FROM raw_fraud_logs
    GROUP BY 1, 2
) AS inner_catalog_query
)

```

-- 3. The Adaptive Assembly

```

SELECT *
FROM unified_scores
LEFT JOIN fraud_strict
    USING (user_id, event_date)
LEFT JOIN fraud_loose
    USING (user_id, event_date)

```

By adding the outer subquery layer, the complexity of the catalog logic is entirely isolated from the user's workflow. The user gets a declarative, self-service interface where they are guaranteed to receive exactly the columns they request, with the parameters they specify, without writing any SQL themselves.

Want to dynamically handle column name collisions?

↔ Yes

Give a detailed plan of this architecture, describing the python project organization (with location of each file), the modules, functions (with its signature and description) etc.

Here is the complete blueprint for the **Configuration-Driven Star Schema Generator**.

This architecture cleanly separates the user configuration (what they want), the catalog (the available logic), and the rendering engine (how it is built). It relies on pure Python and Jinja2, avoiding heavy external frameworks.

1. Project Directory Structure

The repository is divided into three distinct zones: Execution (`src/`), Logic Library (`templates/`), and User Input (`config/`).

```
sql_generator_project/
├── config/
│   └── user_runbook.yml          # The declarative YAML input from the end-user
├── templates/
│   ├── queries/
│   │   └── build_master.sql      # The main Jinja template (Outer Pruning & Joi
│   ├── macros/
│   │   └── spark_normalize.sql  # The unpivot schema-enforcement macro
│   └── catalog/
│       ├── dim_fraud.sql        # The library of parameterized dimension files
│       └── dim_user.sql
├── src/
│   ├── __init__.py
│   ├── parser.py                # Module for parsing and validating YAML
│   └── compiler.py              # Module for managing the Jinja2 environment
├── requirements.txt             # dependencies: PyYAML, Jinja2
└── main.py                      # The primary execution script
```

2. Python Modules & Functions

The Python layer acts strictly as an orchestrator. It reads the state, sets up the templating environment, and triggers the compilation.

Module: `src.parser`

Handles the ingestion and structural validation of the user's YAML configuration.

Function Signature

```
load_runbook(filepath: str) ->
dict
```

Description

Reads the `user_runbook.yml` file from the disk and parses it into a native Python dictionary using `yaml.safe_load()`.

Function Signature

Description

```
validate_config(config: dict) ->  
None
```

Asserts that required top-level keys (`master_pks` , `scores` , `enrichments`) exist and are formatted correctly. Raises a `ValueError` if the user's YAML is structurally invalid.

Module: `src.compiler`

Manages the Jinja2 rendering engine and context injection.

Function Signature

Description

```
setup_env(template_dir: str) ->  
jinja2.Environment
```

Initializes the Jinja environment using `FileSystemLoader` pointing to the `templates/` directory. Disables autoescaping (since we are generating SQL, not HTML) and configures strict undefined variable handling.

```
compile_query(env:  
jinja2.Environment, template_name:  
str, context: dict) -> str
```

Loads the specified template (e.g., `queries/build_master.sql`), injects the Python dictionary as the rendering context, and returns the final compiled SQL string.

Module: `main.py`

The entry point of the application. It orchestrates the flow between the parser and the compiler.

Function Signature

Description

```
main() -> None
```

Defines paths, calls `load_runbook()` , initializes the compiler via `setup_env()` , executes

Function Signature

Description

`compile_query()` , and outputs or writes the final SQL string to a destination (or directly to a Spark session).

3. The Jinja Template Library

These files contain the SQL generation logic. They transform the Python dictionary into the final query string.

A. The Core Macro: `templates/macros/spark_normalize.sql`

Purpose: Enforces the schema and unpivots the fact tables.

Input: The `scores` dictionary and the `master_pks` list.

Logic:

1. Iterates through each score table.
2. Checks if each master primary key exists in the table's specific physical keys. If missing, injects `NULL AS key_name` .
3. Generates the `LATERAL VIEW stack(k, ...)` clause dynamically based on the length of the score mapping array.
4. Appends `UNION ALL` between iterations.

B. The Catalog: `templates/catalog/*.sql`

Purpose: Houses the complex, parameterized business logic for dimensions.

Structure: Standard Spark SQL queries.

Integration: These files act as black boxes. They utilize Jinja variables via the `params` dictionary (e.g., `{{ params.threshold | default(0.5) }}`) injected dynamically during compilation. They output a standard set of keys and a wide array of potential metrics.

C. The Orchestrator: `templates/queries/build_master.sql`

Purpose: The central nervous system of the generator.

Execution Flow:

1. **Fact Generation:** Calls `spark_normalize` to build the `unified_scores` Common Table Expression (CTE).
2. **Enrichment Generation (Outer Pruning):** Iterates over the user's `enrichments`. Sets the `params` context for the loop iteration, uses `{% include %}` to evaluate the catalog file as an inner subquery, and wraps it in an outer `SELECT` that extracts exactly the primary keys and `columns` the user requested.
3. **Adaptive Assembly:** Generates the final `SELECT *` from the fact table. Loops through the enrichments one last time to dynamically calculate the intersection of keys (Fact vs. Dimension) and generates a deduplicating `USING (keys...)` join clause.

4. Execution Sequence

1. The user updates `config/user_runbook.yml` to define their unpivot parameters and requests a dimension from the catalog, specifying exactly which columns and parameter overrides they want.
2. The user executes `python main.py`.
3. `src.parser.load_runbook()` transforms the YAML into a Python dictionary.
4. `src.compiler.setup_env()` builds the Jinja environment, mapping the `templates/` directory.
5. `src.compiler.compile_query()` pushes the configuration state into `build_master.sql`.
6. Jinja evaluates the loops, resolves the catalog includes, performs the column pruning, and mathematically intersects the join keys.
7. The script yields a fully optimized, parameter-injected, single-scan Spark SQL string ready for execution.

Give a overview of this project with its requirements and what may be added or changed as improves

Project Overview